ECE-4270 Computer Architecture

Ekincan Ufuktepe

03/04/2025

Christian Watts

Lab 1

RISC-V Simulator

This simulator loads a RISC-V program into memory and is structured around a fetch-decode-execute cycle. It decodes and executes instructions while updating registers and memory. It also provides debugging tools like register/memory dumps and assembly output.

Custom Definitions

Custom definitions were added to make the identification of operation codes easier

```
// Custom
#define OPCODE_R_TYPE   0x33
#define OPCODE_I_TYPE   0x13
#define OPCODE_LOAD     0x03
#define OPCODE_STORE    0x23
#define OPCODE_LUI      0x37
#define OPCODE_SYSCALL  0x73
#define OPCODE_BRANCH   0x63
#define OPCODE_JAL      0x6F
#define OPCODE_JALR     0x67
#define OPCODE_JR       0x6C
#define SYS_CALL_EXIT   0x0000000C
```

Handle_Instruction:

This function is the main decoding and execution function during run time.

1. At the beginning, we retrieve the current instruction from memory using the current state program counter. This simulates the instruction fetch stage in the CPU.

```
uint32_t instruction = mem_read_32(CURRENT_STATE.PC);
```

2. We decode the instruction to extract the operation code, the destination register, function three and function seven, the two operands, and the immediate value using bit shifting and the provided bitmasks:

```
uint8_t opcode = instruction & BIT_MASK_7;
uint8_t rd  = (instruction >> 7)  & BIT_MASK_5;
uint8_t rs1 = (instruction >> 15) & BIT_MASK_5;
uint8_t rs2 = (instruction >> 20) & BIT_MASK_5;
uint8_t funct3 = (instruction >> 12) & BIT_MASK_3;
uint8_t funct7 = (instruction >> 25) & BIT_MASK_7;
int32_t imm = instruction >> 20;
```

3. For each operation code type, we utilize the included processing functions with our decoded instruction variables and update the program counter:

```
switch(opcode) {
    case OPCODE_R_TYPE: {
        R_Processing(rd, funct3, rs1, rs2, funct7);
        NEXT_STATE.PC = CURRENT_STATE.PC + 4;
        break;
    }
    case OPCODE_I_TYPE: {
        Iimm_Processing(rd, funct3, rs1, imm);
        NEXT_STATE.PC = CURRENT_STATE.PC + 4;
        break;
    }
    case OPCODE_LOAD: {
        ILoad_Processing(rd, funct3, rs1, imm);
        NEXT_STATE.PC = CURRENT_STATE.PC + 4;
        break;
    }
    case OPCODE_STORE: {
        // Immediate is split between bits [11:5] and [4:0]
        uint32_t imm4 = imm & BIT_MASK_5;
        uint32_t imm11 = (imm >> 5) & BIT_MASK_7;
        S_Processing(imm4, funct3, rs1, rs2, imm11);
        NEXT_STATE.PC = CURRENT_STATE.PC + 4;
        break;
    }
    case OPCODE_LUI: {
        NEXT_STATE.REGS[rd] = imm;
        NEXT_STATE.PC = CURRENT_STATE.PC + 4;
        break;
    }
```

4. Including the syscall to exit the program

```
// Special Operations & Sys Call
case OPCODE_SYSCALL: {
    if (instruction == SYS_CALL_EXIT) {
        // If $v0 holds 10, exit the simulation.
        if (CURRENT_STATE.REGS[2] == 10)
            exit(0);
    } else {
        switch(funct3) {
            //MFHI
            case 1:
                NEXT_STATE.REGS[rd] = CURRENT_STATE.HI;
                break;
            //MFLO
            case 2:
                NEXT_STATE.REGS[rd] = CURRENT_STATE.LO;
                break;
            //MTHI
            case 3:
                CURRENT_STATE.HI = CURRENT_STATE.REGS[rs1];
                break;
            //MTLO
            case 4:
                CURRENT_STATE.LO = CURRENT_STATE.REGS[rs1];
                break;
            default:
                printf("Unknown register: %d\n", funct3);
                RUN_FLAG = FALSE;
                break;
        }
    }
    NEXT_STATE.PC = CURRENT_STATE.PC + 4;
    break;
```

Because much of the functionality was already created through utility functions, most of the work came down to analyzing the existing code structure and using accurate bitwise operations to properly decode the instruction.

Print_Program:

Prints the memory location of each instruction and utilizes the print_command function to list the operation, destination register, and operands of the instruction.

```
void print_program() {
    uint32_t address = MEM_TEXT_BEGIN;
    printf("\nProgram Loaded:\n");
    for (int i = 0; i < PROGRAM_SIZE; i++) {
        uint32_t instruction = mem_read_32(address);
        printf("0x%08x: ", address);
        print_command(instruction);
        address += 4;
    }
}
```